Python Nivell 101

Juan Miguel Ramón Tur

Revisió 0.0

Continguts

1. Introducció 5
2. Història del llenguatge *Python* 5
3. *Python*, un llenguatge interpretat 5
   3.1. Avantatges dels llenguatges interpretats 5
   3.2. Desavantatges dels llenguatges interpretats 6
4. La sintaxi 6
   4.1. La indentació del codi 7
   4.2. Estructura general d'un programa 8
5. Variables 9
   5.1. Denominació de variables 9
   5.2. Assignació de valors 10
   5.3. Declaració de variables 10
6. Tipus de dades 11
7. Expressions 12
   7.1. Aritmètiques 13
   7.2. Expressions lògiques 14
   7.3. Els operador lògics ***not***, ***or*** i ***and*** 14
   7.4. Precedència dels operadors 15
8. Interacció bàsica amb l'usuari 17
   8.1. Entrada i sortida per consola 17
9. Control de flux condicional 18
   9.1. Com fer un ***if/else*** amb una sola línia de codi 18
10. Iteracions 19
   10.1. Ús de ***range()*** a les iteracions 19
   10.2. Iterar infinitament! 20
   10.3. Ús de ***break*** i ***continue*** 21
   10.4. Ús de l'***else*** a les iteracions 22
11. Les *funcions* 23
   11.1. *Funcions* integrades 23

11.2. Millorant el codi: *funcions* pròpies: 25
12. Creació de *funcions* pròpies 26
12.1. Paràmetres i *arguments* 26
12.2. Vinculació d'arguments per posició 27
12.3. Vinculació d'arguments per nom 27
12.4. Paràmetres per defecte 27
12.5. Retorn de valors 28
13. Àmbit d'ús de les variables 29
13.1. Variables locals 29
13.2. Variables globals 29
13.3. Variables globals i locals amb el mateix nom 30
13.4. Variables globals dins de les *funcions* 30
14. Gestió d'excepcions 31
14.1. Ús de ***try-except*** 31
14.2. Aixecant excepcions manualment 32
14.3. Tipus d'excepcions integrats 33
14.4. Creació de nous tipus d'excepcions 33
15. Operacions bàsiques amb arxius 34
15.1. Lectura i escriptura d'arxius de text 34
15.2. Lectura i escriptura d'arxius binaris 35
15.3. Modes d'apertura d'arxius 35
15.4. Ús de ***with*** 36
15.5. Mètodes per operar amb arxius 36
15.6. Errors comuns operant amb arxius 37
15.7. Els arxius CSV 38
15.8. Els arxius JSON 38
16. Més sobre els tipus de dades 38
16.1. Tipus mutables i tipus immutables 38
16.2. Anotacions (*hints*) de tipus 38
16.3. Conversió de tipus de dades 38
16.4. Validació de tipus de dades 38

17. Les cadenes en profunditat 38
17.1. Mètodes de les cadenes 39
17.2. Cadenes especials ***f-string*** 39
17.3. Cadenes especials ***raw*** 39
18. Les col·leccions en profunditat 39
18.1. Tipus de col·lecció adequada segons cas d´ús 40
18.2. Rendiment de tuples i llistes 41
18.3. Recorregut de col·leccions 41
18.4. Aplicant ***enumerate()*** en els recorreguts 42
18.5. Cerca d'elements a les col·leccions 42
19. Mòduls i llibreries 43
19.1. *Mòduls* integrats 43
19.2. *Mòduls* externs 43
19.3. Ús del *mòdul* extern ***Colorama*** 43
20. Recomanacions d'estil de programació 44
21. Exercicis breus 44
21.1. amb expressions aritmètiques 44
21.2. amb expressions lògiques 45
21.3. amb condicions 47
21.4. amb iteracions 49
22. Mini-projectes 50
22.1. Optimització d'ús de col·leccions 50
22.2. Generador de frases 50
22.3. Camí de sortida d'un laberint 50

1. Introducció

2. Història del llenguatge *Python*

Python és un llenguatge de programació d'alt nivell, interpretat i multiplataforma. Va ser creat el 1989 per *Guido van Rossum* mentre treballava al Centre per a les Matemàtiques i la Informàtica (CWI) als Països Baixos. La filosofia de *Python* es basa en la llegibilitat del codi i la facilitat d'ús.

El nom *Python* prové de l'afició del seu creador pels programes de televisió britànics de comèdia, específicament de Monty *Python's* Flying Circus.

Python s'ha convertit en un dels llenguatges de programació més populars a causa de la gran quantitat de biblioteques disponibles que faciliten tasques específiques, com ara el processament de dades, l'aprenentatge automàtic, la intel·ligència artificial, la ciència de dades, l'automatització de tasques i el desenvolupament web, entre d'altres.

3. *Python*, un llenguatge interpretat...

Python, com ara *Ruby* i *Javascript* entre d'altres, és un llenguatge de programació interpretat. Els llenguatges de programació interpretats no són executats directament pel maquinari sinó mitjançant un programari auxiliar, que es denomina intèrpret. El codi *Python*, i el dels llenguatges interpretats en general, s'analitza i executa línia per línia, en canvi, els llenguatges compilats, converteixen el codi font complet d'un programari en codi executable abans d'executar-se.

3.1. Avantatges dels llenguatges interpretats

Major facilitat per crear prototips i fer canvis ràpids, ja que no cal compilar el codi abans d'executar-lo.

El codi és més fàcil depurar, pel fet que, en cas d'error, l'intèrpret indicarà, amb major precisió, la línia del codi font on s'ha detectat l'error, el tipus d'error, la variable afectada...

Portabilitat més senzilla perquè no es requereix compilar el codi per executar-lo en diferents plataformes, es pot executar directament.

3.2. Desavantatges dels llenguatges interpretats

L'execució és, en general, més lenta emprant llenguatges interpretats a causa de:

- La sobrecàrrega en temps i altres recursos que suposa executar l'intèrpret.
- La impossibilitat d'optimitzar el codi de forma global, abans de l'execució, ja que el codi s'analitza i executa línia per línia.
- La impossibilitat de realitzar una optimització per una plataforma específica.
- La no detecció de certs errors abans de l'execució, que si detecten els compiladors, pel fet que els intèrprets no analitzen un programari en conjunt sinó línia a línia.
- El codi font és més difícil de protegir contra còpies i alteracions no autoritzades. De vegades pot ser llegit o recuperat fàcilment i en tot cas, als llenguatges interpretats és més viable un procés d'enginyeria inversa que en el cas dels llenguatges compilats.

4. La sintaxi

La sintaxi de *Python* és coneguda per ser clara, concisa i fàcil d'entendre. Algunes de les característiques més destacables són les següents:

- **Indentació**: La indentació s'utilitza per delimitar els blocs de codi, en lloc de fer servir caràcters com ara ***{}*** o ***begin/end***. Això fa que el codi sigui més llegible i fàcil d'entendre. Se mostra un exemple de la indentació al codi següent.

```
if num < 0:
    print("El número ha de ser positiu.")
else:
    factorial = calcular_factorial(num)
    print("El factorial de", num, "és", factorial)
```

- **Comentaris**: Es fa servir el caràcter **#** per indicar un comentari. Això facilita la documentació del codi. La sintaxis se mostra al codi següent.

```
# Aquesta línia és un comentari
```

- **Variables**: No requereix declarar el tipus de les variables, això significa que les variables es poden reutilitzar per emmagatzemar diferents tipus de dades. La sintaxis per crear variables se mostra al codi següent.

```
num1 = 5 # num1 és una variable entera
num2 = "Hola" # num2 és una variable de cadena
```

- **Sentències condicionals**: Les paraules reservades ***if/elif/else*** s'empren per implementar la lògica de control de flux i la sintaxis se mostra al codi següent.

```
if num < 0:
    print("El nombre ha de ser positiu.")
else:
    print("El nombre és positiu.")
```

- **Iteracions** Les paraules reservades ***while*** i ***for*** per implementar la lògica iterativa flux i la sintaxis se mostra al codi següent.

```
contador = 1
while contador <= 10:
    print(contador)
    contador += 1
```

- **Mòduls**: Mòduls i llibreries es poden importar per accedir a funcions i classes addicionals i la sintaxis se mostra al codi següent.

```
import math


print(math.pi) # imprimeix el valor de pi
```

4.1. La indentació del codi

La indentació del codi es basa en la idea de dividir el codi en blocs i indicar les relacions jeràrquiques entre ells. Allunyar un bloc del començament de la línia, és a dir, indentar-lo, significa establir una relació jeràrquica. El bloc de codi desplaçat cap a la dreta passa a estar supeditat a un bloc de jerarquia superior, que es trobarà més prop del començament de la línia.

Cal destacar que, els dos punts són utilitzats per indicar que comença un nou bloc de codi de nivell jeràrquic inferior i, per tant, depenent de la línia on hi ha els dos punts.

Al codi següent trobem una iteració, de tipus ***for***, de la que depèn jeràrquicament un petit bloc amb una sola instrucció...

```
for i in range(10):
    print(i)

print("Iteració finalitzada")
```

A la iteració ***for*** de l'exemple la sentències indentada s'executarà en cada iteració, mentre que la sentència no indentada s'executarà una vegada completada la iteració.

4.2. Estructura general d'un programa

Un programa escrit en *Python* està compost per un conjunt d'instruccions que especifiquen les operacions que el programa ha de realitzar. Les instruccions es poden dividir principalment en quatre blocs:

- **Declaracions d'importació**: Tenen per objectiu reutilitzar altres fragment de codi, anomenats mòduls o paquets que contenen funcions i classes que són útils per al programari que estam creant.
- **Funcions o classes**: Bloc de codi on se defineixen totes les funcions.
- **Declaracions de variables**: Bloc d'instruccions on se creen les variables globals. Això se realitza mitjançant operacions d'assignació de valors.
- **Programa principal**: Bloc d'instruccions especifiquen les operacions que el programari ha de dur a terme.

Els programaris poden tindre una estructura més elaborada i, com no, mes desordenada, o seguir un ordre diferent, però recomano començar emprant aquest ordre i estructura...

Tot seguit se mostra un exemple de codi d'un programari complet, amb una estructura senzilla amb l'organització recomanada!

```
 ### Importació de mòduls ###
import math


### Funcions ###
def es_primer(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n))+1):
        if n % i == 0:
            return False
    return True
```

```
### Declaracions de variables  ###
num = int(input("Introdueix un nombre: "))


### Programa principal ###
if es_primer(num):
    print(num, "és un nobmre primer.")
else:
    print(num, "no és un nombre primer.")
```

5. Variables

En programació, les variables són espais reservats a la memòria del computador on es poden emmagatzemar valors. Les variables es poden considerar com a contenidors o caixes que contenen valors. Aquests valors es poden fer servir per fer càlculs, realitzar comparacions, passar dades entre diferents parts del programa i moltes altres operacions.

La forma d'accedir al valor d'una variable és mitjançant un nom, que el programador li assigna i que identifica de forma inequívoca la variable. Fent ús d'aquest nom es pot recuperar el valor emmagatzemat i també és possible modificar-lo durant l'execució del programa.

5.1. Denominació de variables

Les variables tenen un nom o identificador que es fa servir per referir-se a elles. Aquests noms, que el programador pot decidir segons el seu criteri, han de seguir les qualcunes regles:

- Han de començar amb una lletra o un guió baix (*underscore*).
- Poden contenir lletres, dígits i guions baixos.
- Són sensibles a majúscules i minúscules, per tant, la variable x i la variable X són diferents (*case sensitive*).
- No poden ser paraules reservades del llenguatge, com ara if, else, while, for, etc.

És important tenir present que els noms que s'assignen a les variables ens han d'ajudar a fer que el nostre codi sigui més llegible i fàcil de mantenir. Per aquest motiu els noms de les variables han de ser significatius reflectint el seu contingut i objectiu.

5.2. Assignació de valors

L'assignació de valors a variables és una operació bàsica en programació que permet donar un valor a una variable específica. L'operació d'assignació consisteix en assignar un valor a un nom o identificador que representa una variable.

En general, en la majoria dels llenguatges de programació, l'assignació es fa mitjançant el signe d'igualtat **=** , que actua com operador d'assignació. El valor de la dreta del signe d'igualtat és assignat a la variable de l'esquerra.

```
# Declarem una variable anomenada "x" i li assignem el valor 10
x = 10
```

En aquest exemple, hem creat una variable anomenada "x" i li hem assignat el valor 10. A partir d'aquest punt, la variable "x" contindrà el valor 10, i aquest valor es pot utilitzar en el programa per fer càlculs, comparacions o altres operacions.

Es poden fer assignacions més complexes, com ara assignacions múltiples en una sola línia o assignacions basades en expressions, com al codi que segueix.

```
# Declarem tres variables i els assignem valors diferents en
una sola línia
a, b, c = 1, 2, 3
```

5.3. Declaració de variables

Declarar una variable és sinònim de crear una variable. El procés de crear o declarar una variable suposa assignar un espai de memòria per guardar un valor i associar un nom a aquest espai de memòria. Des del moment de la creació de la variable el programari queda assabentat de l'existència de la nova variable i, per tant, pot ser emprada si es necessita.

Les variables es declaren simplement assignant un valor a un nom o identificador. A diferència d'altres llenguatges de programació, no és necessari declarar explícitament el tipus de dada que contindrà la variable.

El següent codi mostra alguns exemples de com declarar variables.

```
# Assignació simple
x = 42

# Assignació múltiple
a, b, c = 1, 2, 3

# Assignació a través d'una expressió
```

```
d = 2 * a + b

# Assignació amb una cadena de text
nom = "Jordi"

# Assignació amb una llista
llista = [1, 2, 3]

# Assignació amb un diccionari
dicc = {"clau1": "valor1", "clau2": "valor2"}

# Assignació amb una tupla
tupla = (1, 2, 3)

# Assignació amb un valor de None
z = None
```

6. Tipus de dades

Python té diversos tipus de dades integrats que es poden utilitzar per emmagatzemar i treballar amb diferents tipus d'informació. Alguns dels tipus de dades més comuns són:

- **Nombres sencers** (***int***): Són nombre sense decimals, positius i negatius. A *Python*, a diferencia de la majoria de llenguatges, no hi ha limitació en el valor màxim que es pot emmagatzemar emprant aquest tipus. Exemple:

  ```
  # sencer
  a = 5
  ```

  Pel que fa al valor màxim, el nombre calculat al següent exemple és enorme, concretament té 22.006 dígits... i se pot emmagatzemar dintre d'una variable sencera

  ```
  x = 43423**4745
  print(len(str(x))) # Sortida 22006
  ```

- **Nombres en coma flotant** (***float***): Representa nombre positius o negatius amb decimals, és a dir, nombres reals. Exemple:

  ```
  # punt flotant
  b = 3.14
  ```

- **Boolea (*bool*)**: És un tipus de dada que permet emmagatzemar dos valors ***True*** o ***False***.

```
a = True
b = 3>5
```

- **Cadena** (***str***): Representa una cadenes de caràcters. Les cadenes es creen utilitzant cometes simples o dobles. Exemple:

```
a = "Hola a tothom!"
b = 'Python és genial'
```

- **Llista** (***list***): És un tipus de dades que permet emmagatzemar una col·lecció d'elements en un ordre determinat. Es creen utilitzant claudàtors ***[ ]***. Exemple:

```
a = [1, 2, 3, 4, 5]
b = ["ca", "gat", "serp"]
```

- **Tuples**: Se semblen a les llistes, però són immutables, és a dir, un cop creades, no es poden modificar. Es creen fent servir parèntesis ***( )***. Exemple:

```
a = (1, 2, 3, 4, 5)
b = ("ca", "gat", "serp")
```

- **Conjunt** (***set***): És un tipus de dada que permet emmagatzemar una col·lecció d'elements sense ordre i sense duplicats. Es poden crear utilitzant claus ***{}***. Exemple:

```
B = {1, 2, 3, 4, 5}
b = {"ca", "gat", "serp"}
```

- **Diccionari**: És un tipus de dada que permet emmagatzemar una col·lecció d'elements en un format clau-valor. Es poden crear utilitzant claus ***{}*** i dos punts ***:***. Exemple:

```
a = {"nombre": "Joan", "edad": 30}
b = {1: "ca", 2: "gat", 3: "serp"}
```

7. Expressions

Les expressions són una combinació de valors constants, variables, operadors, funcions, parèntesis i altres subexpressions. Cada expressió en ser avaluada produeix un únic resultat. Aquest resultat pot ser assignat a una variable, emprat com operand a una expressió de nivell superior o amb moltes altres finalitats. Les expressions es fan servir per realitzar càlculs matemàtics, lògics o d'altres

tipus i són fonamentals en l'escriptura dels programaris, sigui en *Python* o en quasi qualsevol llenguatge de programació.

Segons el tipus de la dada resultant en avaluar-se, les expressions, es poden classificar en distints tipus dels quals destaquem els següents:

- **Expressions aritmètiques**: Permeten realitzar operacions matemàtiques com ara suma, resta, multiplicació i divisió. Exemple:

```
2 + 3 avalua a 5.
```

- **Expressions lògiques**: Són utilitzades per comparar valors de qualsevol mena i efectuar operacions lògiques. Retornen un valor booleà.

```
3 > (2/0.5) avalua a True.
```

- **Expressions de cadenes**: Retornen una cadena. En moltes ocasions la creació de la nova cadena s'obté combinant altres cadenes, com al següent exemple:

```
"Hola " + "Mundo" avalua a "Hola Mundo".
```

7.1. Aritmètiques

Permeten realitzar operacions matemàtiques com sumar, restar, multiplicar, potenciar i dividir entre d'altres. En ser avaluades el resultat pot ser un nombre sencer o amb decimals, es a dir, de coma flotant.

Els operadors aritmètics més comuns que s'utilitzen per fer aquestes operacions són: **+**, **-**, *****, **/**, **%**, ****** (potencia) i **//** (divisió sencera). Exemple:

```
a = 5
b = 2

# Mòdul
c = a % b
print(c) # 1

# Potencia
c = a ** b
print(c) # 25

# Divisió sencera
c = a // b
print(c) # 2
```

7.2. Expressions lògiques

Les expressions lògiques permeten fer d'operacions entre valors booleans o avaluar la relació entre valors d'altres menes, però en calcular-se el resultat és sempre booleà (veritable o fals). Aquestes expressions poden incloure operacions relacionals (igualtats, desigualtats, inclusions...,) o operacones lògiques com són les negacions, conjuncions i disjuncions.

Python té un conjunt d'operadors, relacionals i lògics, i el que tenen en comú és en aplicar-se, el resultat és sempre booleà. Aquests operadors són: **==**, ***!=***, **>**, **>=**, **<**, **<=**, ***not***, ***and*** i ***or***. Igual que en les expressions aritmètiques es poden fer servir parèntesi per indicar-li a l'intèrpret la prioritat en les operacions.

Vegem un exemples de expressions lògiques:

```
a = 5
b = 2

# Igualtat
c = (a == b)
print(c)  # False

# Desigualtat
c = (a != b)
print(c)  # True

# Negació
c = not (a == b)
print(c)  # True

# Disjunció
c = (a > b) or (a == b)
print(c)  # True
```

7.3. Els operador lògics ***not***, ***or*** i ***and***

Són operadors lògics que s'utilitzen per efectuar operacions amb valors booleans (veritable o fals):

- **not**: Inverteix el valor booleà d'una sola expressió, ja que és un operador unari. Si l'expressió és vertadera, el resultat serà fals i, si l'expressió és falsa, el resultat serà vertader. Exemple:

```
a = True
b = not a
print(b) # False
```

- **or:** Avalua dues expressions lògiques i torna veritable si almenys una de les expressions és veritable. Si les dues expressions són falses, torna fals. Exemple:

```
a = True
b = False
c = a or b
print(c) # True
```

- **and:** Avalua dues expressions lògiques i torna veritable si les dues expressions són veritables. Si almenys una de les expressions és falsa, torna fals. Exemple:

```
a = True
b = False
c = a and b
print(c) # False
```

7.4. Precedència dels operadors

És important tenir en compte que el resultat d'una expressió pot canviar segons l'orde en què es facin les operacions i càlculs. Això podria lloc a errors o comportaments inesperats del programari, en conseqüència, és de la màxima importància conèixer les regles per preveure l'ordre d'execució de les operacions per obtenir el resultat.

Hi ha un ordre de precedència establert (prioritat) per als operadors que determina l'ordre en què s'avaluen les operacions en una expressió. Això és similar a com funcionen les operacions matemàtiques en matemàtiques... Aquí hi ha una llista ordenada segons prioritat d'aplicació dels operadors (*https://docs.python.org/es/3/reference/expressions.html#operator-precedence*):

Operador	**Descripció**
(expressions...) *[expressions...]* *{key: value...}* *{expressions...}*	Expressió d'enllaç o entre parèntesis, llista, diccionari i conjunt
x[index], x[index:index], x(arguments...), x.attribute	Subscripció, segmentació, invocacions, referència d'atribut
**	Potenciació

Operador	Descripció
+x, -x, ~x	NOT positiu, negatiu, bit a bit
**, @, /, //, %*	Multiplicació, multiplicació de matrius, divisió, divisió de sencera, resta
+, -	Addició i resta
<<, >>	Desplaçaments de bits
&, `, |	AND, XOR, OR bit a bit
in, not in, is, is not, <, <=, >, >=, !=, ==	Comparacions, comprovacions de pertinència i d'identitat
not , and, or	Operadors lògics
if - else	Expressió condicional
lambda	Funció anònima
=	Assignació

Així, si hi ha operadors amb la mateixa precedència, s'avaluen de l'esquerra a la dreta.

Si, a una expressió, necessiteu canviar l'ordre d'avaluació (predeterminat) de les operacions has de fer servir parèntesis i així establir un ordre diferent segons les necessitats.

Exemple amb expressió aritmètica: La precedència de la multiplicació és superior a la precedència de l'addició. Per aquest motiu es multiplica 4 * 5 abans de sumar 3. Si el que desitja és sumar primer 3 i 4 s'han d'emprar parèntesis.

```
# Precedències per defecte
resultat = 3 + 4 * 5
print(resultat) # 23

# Amb parèntesis
resultat = (3 + 4) * 5
print(resultat) # 35
```

Exemple amb expressió lògica: La disjunció (or) té prioritat sobre la conjunció (and). Per tant, s'avalua primer *"False or True"* i després ja s'aplica l'operador

"and"... En cas de voler avaluar primer *"True and False"* s'han d'emprar parèntesis per indicar l'intèrpret la prioritat correcta.

```
# Precedència per defecte
resultat = (True and False) or True
print(resultat) # True

# Amb parèntesis
resultat = True and (False or True)
print(resultat) # False
```

8. Interacció bàsica amb l'usuari

Python es pot executar tant en entorns textuals de consola com en entorns gràfics de finestres. Això significa que és possible desenvolupar aplicacions que tinguin una interfície atractiva amb finestres, botons, caixes de text i altres elements gràfics, o bé que la interfície sigui una consola, basada en línia d'ordres, a priori, molt més rudimentària...

Per crear interfícies gràfiques, es fan servir llibreries com *Tkinter*, *PyQt* o *wxPython*. Aquestes llibreries proporcionen les eines necessàries per crear finestres, botons, menús i altres elements d'interfície de l'usuari. Les deixem per més endavant...

Les interfícies basades en la consola, per lo general, no requereixen importar llibreries. Proporcionen una interfície basada en pur text, on l'usuari pot introduir dades i veure els resultats. Són molt utilitzades per desenvolupadors i ja que permeten crear interfícies senzilles de forma ràpida però aleshores poc atractives. Per aquest motiu no se solen emprar per crear solucions per usuaris finals que no tinguin un perfil tècnic.

8.1. Entrada i sortida per consola

Per interactuar amb l'entrada/sortida per consola es fan servir les funcions següents:

- ***input()***: Permet llegir una línia d'entrada del teclat i retorna la cadena de caràcters que s'ha llegit. Atenció sempre retorna una cadena de caràcters tot i que, a la consola, teclegem nombres.

```
nom = input("Introdueix el teu nom: ")
```

- ***print()***: Permet escriure a la sortida estàndard (normalment la consola) una cadena de caràcters o altres tipus de dades que es converteixen automàticament a una cadena.

```
print("Hola, " + nom + "!")
```

9. Control de flux condicional

La instrucció de control de flux condicional és una estructura de control que permet executar un bloc de codi només si es compleix una determinada condició.

La sintaxi bàsica d'una instrucció de control de flux condicional és la següent:

```
if condició:
    # Bloc de codi que s'executarà si la condició és verdadera
else:
    # Bloc de codi que s'executarà si la condició és falsa
```

On la condició és una expressió lògica i que, per tant, es pot avaluar com a certa o falsa.

Les sentencies de control de flux condicional poden portar clàusules ***elif***, que s'usen en necessitar avaluar diverses condicions i només executar el bloc de codi corresponent a la primera condició verdadera...

```
if condició 1:
    # Bloc de codi que s'executarà si la condició 1 és verdadera
elif condició 2:
    # Bloc de codi que s'executarà si la condició 1 és verdadera
else:
oque se ejecuta si no se cumple ninguna de las condiciones
anteriores
```

9.1. Com fer un ***if/else*** amb una sola línia de codi

Python, com també Java i altres llenguatges, té una estructura sintàctica que permet substituir un ***if/else*** tradicional amb una sola línia de codi. Aquestes estructures sintàctiques empren, els denominats en moltes ocasions operadors ternaris, tot i no ser-ho estrictament!

En el cas de *Python* la sintaxi d'aquest "operador ternari" és:

```
valor_a_assignar = valor_si_és_cert if condició else
valor_si_és_fals
```

Això permet assignar a ***valor_a_assignar*** el valor de ***valor_si_és_cert*** si ***condició*** és ***True***, o ***valor_si_és_fals*** si ***condició*** és ***False***. Per exemple:

```
x = 10
resultat = "major que 5" if x > 5 else "menor o igual a 5"
print(resultat)  # sortida: "major que 5"
```

10. Iteracions

Als llenguatges de programació els cicles, iteracions, bucles o *loops* (com també es coneixen), són estructures de control del flux que permeten executar un bloc de codi repetidament. Hi ha dos tipus d'estructures iteratives:

- Els bucles ***for***: Són utilitzats per a iterar sobre una seqüència d'elements, com ara una llista, una tupla o un rang de números. Aquest tipus d'iteració està molt indicat quan sabem el nombre de vegades que volem repetir el codi o quan volem iterar sobre una seqüència específica, com ara una llista. Exemple:

```
# Exemple d'un cicle for
for i in range(10):
    print(i)
```

- Els bucles ***while***: Són utilitzats per a iterar mentre es compleix una condició. Aquest tipus de cicle se fa servir quan no sabem el nombre de vegades que volem repetir el codi o quan volem iterar mentre es compleix una condició específica. Exemple:

```
i = 0
while i < 10:
    print(i)
    i += 1
```

En general, l'estructura ***for*** és més indicada quan sabem el nombre de vegades que volem repetir el codi o quan volem iterar sobre una seqüència específica, mentre que l'estructura ***while*** és més indicada quan no sabem el nombre de vegades que volem repetir el codi o bé quan volem iterar mentre es compleix una condició específica.

10.1. Ús de ***range()*** a les iteracions

És una funció que s'utilitza per generar una seqüència de números. És molt usat en conjunt amb els cicles ***for*** per iterar sobre un rang de números. La seva sintaxi la següent:

```
range(start, stop, step)
```

On:

- **start:** Nombre des del qual es començarà la seqüència (per defecte és 0)
- **stop:** Nombre fins al qual es generarà la seqüència (no s'inclou aquest número)
- **step:** Valor que se sumarà a cada iteració (per defecte és 1)

Aquí teniu qualcuns exemples.

```
# Mostra  una seqüència de nombres del 0 al 9
for i in range(10):
    print(i)

# Mostra  una seqüència de nombres del 0 al 9 de 2 en 2
for i in range(0, 10, 2):
    print(i)

# Mostra  una seqüència de nombres  del 10 al 1
for i in range(10, 0, -1):
    print(i)
```

10.2. Iterar infinitament!

Les iteracions infinites són en moltes ocasions un problema, comú en iniciar-se en la programació. Es produeixen quan una instrucció iterativa no té una condició de permanència que arribi a valer ***False*** en cap ocasió i en aquest cas les instruccions interiors del bucle s'executaran repetidament i per sempre. Això ens podria portar a una aplicació que no respon, a una sobrecàrrega del sistema i, finalment, i fins i tot a un col·lapse del sistema per consum excessiu de recursos.

Considera el següent codi.

```
while True:
    print("This is an infinite loop")
```

Aquest bucle no té cap manera de sortir, ja que la condició (certament minimalista) sempre és verdadera. Com a resultat, el bucle s'executarà indefinidament i el programa no acabarà mai d'executar-se.

Per evitar bucles infinits per error, és important garantir que la condició de permanència està ben escrita, és a dir, que

avaluï a False clarament en donar-se les circumstàncies en les quals volem deixar d'iterar.

Com hem vist el més normal és que els bucles infinit es produeixin a les estructures de tipus ***while***, però a Python existeix la possibilitat de crear bucles infinits amb estructures tipus ***for***. Per tant, també cal estar atent amb els cicles de tipus ***for*** per tal de no crear bºucles infinits no desitjats. Exemple:

```
from itertools import *

# Bucle infinit
# Cycle es un generador de seqüències cícliques infinit
for i in cycle("ABC"):
    print(i)

# Bucle infinit
# count() és un generador de seqüències de nombres infinit
for i in count(10):
    print(i)
```

NOTA: El mòdul ***itertools*** és un mòdul de la llibreria estàndard de Python que proporciona un conjunt d'eines útils per treballar amb iterables, és a dir, objectes que es poden iterar com ara llistes, tuples, conjunts i generadors. Aquest mòdul conté diverses funcions que són útils per a la creació i manipulació d'iterables.

10.3. Ús de *break* i *continue*

La sentència, ***break*** i la sentència ***continue*** són dos comandaments comuns en la programació de bucles, tot i que no es recomana el seu ús amb excés, ja que pot afectar negativament a la llegibilitat del codi. El seu objectiu és canviar la seqüència normal d'execució d'instruccions.

- ***break***: Aquesta sentència és utilitzada per sortir d'un bucle i continuar amb l'execució del codi següent fora del bucle. Això és útil quan volem deixar de recórrer un bucle abans que arribi al seu final, per exemple si hem trobat el que estàvem buscant o si hem arribat a una condició que impedeix que el bucle continuï.

  Al codi següent si *nombre* és igual a 3, el bucle s'interromp mitjançant una sentència ***break***. Després de la interrupció, el bloc de codi que ve a continuació del bucle s'executarà i mostrarà el missatge *"Iteració interrompuda amb un break"*

```
nombres = [1, 2, 3, 4, 5]
for nombre in nombres:
    if nombre == 3:
```

```
        break
    print(nombre)
print("Iteració interrompuda amb un break")
```

- ***continue***: S'empra per saltar a la següent iteració d'un bucle, sense executar el codi restant d'aquesta iteració (el codi que hi ha desprès del ***continue***).

  Al codi següent, el bucle for recorre la llista *nombres*. Si *nombre* és igual a 3, la sentència ***continue*** fa que la següent iteració comenci sense executar el codi restant d'aquesta iteració. Després de la interrupció, el codi de fora del bucle s'executarà i mostrarà el missatge *"Fi del bucle"*

```
nombres = [1, 2, 3, 4, 5]
for nombre in nombres:
    if nombre == 3:
        continue
    print(nombre)
print("Fi del bucle")
```

10.4. Ús de l'***else*** a les iteracions

L'ús de l'expressió ***else*** en els bucles té una funció diferent de la que es pot esperar. En aquest cas el bloc de codi associat a l'expressió ***else*** d'un bucle s'executa quan s'han recorregut tots els elements de la seqüència que s'està recorrent o bé si és un bucle condicional s'executa quan la condició del bucle esdevé falsa.

Això significa que, si el bucle no s'atura prematurament amb una instrucció ***break***, el bloc de codi associat a l'***else*** s'executarà, però també el contrari, és a dir, si un bucle finalitza mitjançant un ***break,*** el bloc de codi associat a l'else es botarà.

```
# Exemple amb for
nombres = [1, 2, 3, 4, 5]
for nombre in nombres:
    print(nombre)
else:
    print("S'han recorregut tots els elements de la llista")


# Exemple amb while
i = 0
while i < 5:
    print(i)
    i += 1
else:
```

```
    print("La condició del bucle ja no és certa")
```

11. Les *funcions*

El concepte de funció és una idea fonamental en matemàtiques que s'utilitza per descriure la relació entre un conjunt d'entrades (conegudes com a "arguments" o "paràmetres") i un conjunt de sortides (conegudes com a "resultats").

En programació, les funcions també tenen aquesta mateixa idea bàsica. Una funció pren una o més entrades, realitza algun càlcul o tasca i segons els valors subministrats com a entrada, retorna una sortida o resultat.

Per exemple, una funció matemàtica com ara f(x) = x^2 és una funció que pren un valor d'entrada (x), du a terme un càlcul (elevar x al quadrat) i retorna un resultat (x^2). Aquesta funció, que realitza aquest càlcul concret, podria escriure-se així:

```
def quadrat(x):
    return x ** 2
```

Les funcions es denominen funcions perquè són una eina per descriure i implementar relacions entre entrades i sortides, de la mateixa manera que les funcions matemàtiques descriuen aquesta mateixa relació. De manera més informal podríem dir que la denominació de funció respon a que són blocs de codi que estan orientats, tenen com a objectiu, realitzar una funció o tasca molt concreta!

11.1. *Funcions* integrades

Les funcions integrades són funcions que ja estan disponibles a la llibreria estàndard de Python. Per gaudir d'aquestes funcions no és necessari crear codi addicional, ni tan sols han de ser importades des d'un mòdul extern. Això significa que es poden utilitzar directament en el codi sense necessitat de definir-les primer.

Per utilitzar una funció integrada, només cal escriure el nom de la funció seguit dels seus arguments entre parèntesis. Els arguments són els valors necessaris per a l'execució de la funció.

Al següent codi es mostren exemples d'ús de funcions integrades amb els seus arguments.

```
# Funció integrada print()
print("Hola món!")
```

```python
# Funció integrada len()
llista = [1, 2, 3, 4, 5]
llista_len = len(llista)
print(llista_len)

# Funció integrada str()
num_enter = 42
num_enter_str = str(num_enter)
print(num_enter_str)
```

A més, és important destacar que les funcions integrades tenen documentació disponible també integrada, que s'obté a través de la funció ***help()***. Això és útil per entendre com fer servir les funcions i per conèixer les opcions i arguments disponibles.

Per exemple, per aconseguir una descripció a sobre la funció ***print(),*** inclosos els arguments disponibles i exemples d'ús es pot executar el següent codi.

```python
help(print)
```

Python té moltes funcions integrades i algunes de les més comunes són:

print()	Imprimeix un text a la consola.
len()	Obté la longitud d'un objecte.
type()	Obté el tipus d'un objecte.
range()	Genera una seqüència d'enters.
input()	Obté una entrada de l'usuari.
int()	Converteix una cadena a un enter.
str()	Converteix un objecte a una cadena.
list()	Converteix un objecte iterable a una llista.
tuple()	Converteix un objecte iterable a una tupla.
dict()	Crea un diccionari.

Aquí tens alguns exemples d'ús de funcions integrades.

```
nom = "Anna"
llargada_nom = len(nom)
print(llargada_nom)  # Imprimirà 4

numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
maxim = max(numeros)
minim = min(numeros)
print(maxim, minim)  # Imprimirà 9 i 1

numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
ordenats = sorted(numeros)
print(ordenats) # Imprimirà [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]

numero = -3
absolut = abs(numero)
print(absolut)  # Això imprimirà 3
```

11.2. Millorant el codi: *funcions* pròpies:

La definició de funcions facilita la *reutilització* de codi, ja que les funcions permeten encapsular un bloc de codi i executar-lo diverses vegades en diferents parts del programa sense haver de tornar a escriure-lo. Això estalvia temps i esforç al desenvolupador.

Les funcions també habiliten la possibilitat de crear codi modular. La *modularitat* fa referència a dividir un problema en parts més petites i manejables, i les funcions són una forma de dividir el codi en parts lògiques i independents i amb una mida tan petita com vulguem o necessitem. La modularitat ens aporta la capacitat d'organitzar un codi, ordenar-lo seguint una certa arquitectura, i si el programari és molt extens o complexa aquesta qualitat esdevé molt important.

Les funcions milloren la *llegibilitat* i comprensió del codi. En encapsular un bloc de codi en una funció, se li pot donar un nom significatiu i es poden entendre millor el objectius i les tasques que realitza aquesta funció. Una funció pot ser estudiada independentment d'altres funcions i funció a funció se pot arribar a comprendre de forma global un programari.

Les funcions ens permeten treballar d'una forma més enfocada a la resolució d'una única problemàtica, amb certa independència del context. Per aquest motiu, cada funció pot ser dissenyada i provada de forma aïllada, cosa que millora moltes tasques com per exemple la depuració i el manteniment del codi.

12. Creació de *funcions* pròpies

Per definir una funció es fa servir la paraula reservada ***def***, seguida del nom de la funció i els paràmetres entre parèntesis. Tot seguit, s'escriu el bloc de codi associat a la funció, alineat just sota el nom de la funció.

```
def nom_funcio(parametre_1, parametre_2, ...):
    # Escriure aquí el bloc de codi associat
```

Tot seguit es crea una funció anomenada *saludar* que té un paràmetre el *nom* i desprès s'invoca amb un argument, "*Joan*".

```
def saludar(nom):
    print ("Hola, " + nom)

saludar("Joan")
```

12.1. Paràmetres i *arguments*

Els paràmetres i els arguments són termes molt relacionats amb les funcions (i mètodes), però ... què són exactament els paràmetres i que són els arguments?

- Els *paràmetres* són les variables que es declaren a la definició d'una funció i que s'usen per emmagatzemar valors que s'han d'especificar en el moment de la invocació de la funció. Aquestes variables (locals), ja inicialitzades, es poden utilitzar per fer càlculs i operacions al bloc de codi associat a la funció.

- Els *arguments* són valors que s'especifiquen en el moment d'invocar una funció. Aquests valors, són assignats als paràmetres (variables locals) corresponents de la funció.

Des del punt de vista de la sintaxi, en definir una funció, els paràmetres es declaren entre parèntesis, just després del nom de la funció, separats per comes, com veiem al següent fragment de codi.

```
def suma(a, b):
    return a + b
```

Com dèiem, els arguments s'especifiquen en el moment d'invocar la funció, i es col·loquen dins dels parèntesis, també separats per comes, tal com es mostra al codi següent. Al codi següent, concretament, la funció anomenada sumar és invocada amb els arguments (valors) de tipus numèric 3 i 4.

```
resultado = suma(3, 4)
```

12.2. Vinculació d'arguments per posició

La vinculació per posició d'arguments a paràmetres és una manera senzilla i comuna de passar arguments a una funció. Significa que els arguments són assignats als paràmetres de la funció en ordre posicional.

```
def sa_meva_funcio(a, b, c):
    print(a, b, c)

sa_meva_funcio(1, 2, 3)
```

Els valors *1, 2,* i *3* són assignats als paràmetres *a, b,* i *c* respectivament, tenint en compte sols la posició dels arguments i la posició dels paràmetres, és a dir, el primer argument s'associa al primer paràmetre, el segon argument al segon paràmetre i així successivament.

12.3. Vinculació d'arguments per nom

La vinculació d'arguments per nom és una altra manera de passar arguments a una funció. En aquest cas, en lloc d'associar els arguments a paràmetres per posició, s'associen mitjançant els seus noms.

Al següent codi ambdues invocacions de la funció mostrarien el mateix resultat per la consola...

```
def sa_meva_funcio(a, b, c):
    print(a, b, c)

sa_meva_funcio(a=1, b=2, c=3)
sa_meva_funcio(c=3, b=2, a=1)
```

Un avantatge de la vinculació d'arguments per nom és que els arguments es poden passar en qualsevol ordre.

12.4. Paràmetres per defecte

Els paràmetres també poden tenir valors per defecte, de manera que no cal passar-los necessàriament en cada crida a la funció. Si un argument no es passa, la funció farà servir el valor per defecte.

Al següent fragment de codi paràmetres *b* i *c* s'iniciaran amb valors per defecte que són *2* i *3* respectivament, ja que s'invoca la funció amb només un argument.

```
def sa_meva_funcio(a, b=2, c=3):
    print(a, b, c)

sa_meva_funcio(a=1)
```

12.5. Retorn de valors

Es habitual que funció retorni els resultats produïts al seu bloc de codi de la mitjançant la instrucció ***return***. El valor que es retorna, per exemple, es pot emmagatzemar a una variable o utilitzar a una expressió.

EL codi que segueix crea una funció anomenada "sumar" que té dos paràmetres "a" i "b". Dins de la funció es retorna el resultat de la suma dels dos paràmetres. Després es crida a la funció "suma" i li passem com a arguments els valors 3 i 4, el resultat de la suma es desa a la variable "resultat" i s'imprimeix.

```
def sumar(a,b):
    return a+b

resultat = sumar(3,4)
print(resultat)
```

Una funció també pot retornar múltiples valors. En realitat el que és fa es retornar una tupla amb tants items com valors se necessita retornar. A continuació figura un codi on hi ha una funció anomenada *dividir* quye rep dos arguments *a* i *b*, i retorna el *quocient* i el *residu* de la divisió. Això es fa retornant una tupla de dos valors a la instrucció ***return***.

```
def dividir(a, b):
    quocient = a // b
    residu = a % b
    return quocient, residu
```

Es pot accedir als valors retornats per la funció utilitzant la descompressió de tupla, que és una sintaxi que permet assignar cada valor de la tupla a una variable diferent. A continuació, un exemple:

```
q, r = dividir(10, 3)
print("Quocient:", q)
print("Residu:", r)
```

13. Àmbit d'ús de les variables

En programació, un *àmbit* o *scope* d'una variable és la regió del programa on la variable pot ser accedida i utilitzada. De vegades, l'àmbit de les variables també es denomina *visibilitat* de les variables.

Els àmbits de les variables són importants per evitar col·lisions o interferències de noms de variables. També permeten garantir que les variables estiguin disponibles només quan es necessitin, així es pot evitar que certes parts d'un programa en faixin mal ús generant errors, que sovint, són molt difícils de diagnosticar.

A Python, hi ha diferents àmbits per a les variables, més concretament les variables poden ser locals o globals. L'àmbit de les variables es determina per la posició en la qual és declarada al codi.

13.1. Variables locals

Les variables declarades dins d'una funció i els paràmetres són sempre locals. Les variables locals només són accessibles des de dins de la funció on s'han creat. Si intentes accedir a aquestes variables des de fora de la funció, obtindràs un error, com ocorre al codi que segueix...

```
def sa_meva_funcio():
    x = 5 # Variable local
    print(x)

sa_meva_funcio() # Output: 5

print(x) # Output: NameError: name 'x' is not defined
```

13.2. Variables globals

Les variables definides fora de totes les funcions i classes són sempre globals. Aquest tipus de variables són accessibles des de qualsevol lloc del programa tant per emprar el valor que emmagatzemen com per modificar-lo.

El següent codi mostra com declarar una variable global i com emprar el seu valor. Cal observar que la variable *x* en cap moment s'ha declarat dintre de la funció ja que no hi ha cap assignació.

```
def sa_meva_funcio():
    print("Valor de x a la funció", x)
```

```
# Declaració variable global x
x = 10

sa_meva_funcio()
print("Valor variable global x", x, "\n")
```

13.3. Variables globals i locals amb el mateix nom

Si es defineix una variable amb el mateix nom en un àmbit local i global, la variable local té prioritat dins de l'àmbit de la funció.

Al codi següent definim una funció, amb una sola línia de codi, amb una assignació on canviem el valor de la variable *x*. Es molt important apreciar que en realitzar aquesta assignació NO està modificant el valor de la variable global *x* sinó que està declarant una nova variable local que també s'anomena *x*.

```
def sa_meva_funcio():
    x = "Hola" # Declaració variable local
    print("Valor de x a la funció", x) # Mostra "Hola"

# Declaració variable global x
x = 10

sa_meva_funcio()
print("Valor variable global x", x, "\n") # Mostra 10
```

ATENCIÓ: En cas de que la coincidència de noms creï ambigüitat la prioritat es considerar la variable com a local.

13.4. Variables globals dins de les *funcions*

Per fer servir una variable global des d'una funció cal fer lo següent:

- Definir la variable fora de la funció. Així, la variable esdevindrà una variable global que es pot accedir des de qualsevol part del codi.
- Indicar, indicar que, a la funció, s'utilitzarà la variable global amb la paraula clau ***global***. Això permet a la funció accedir a la variable global i modificar el seu valor.

```
def sa_meva_funcio():
    global x  # Indicació que es farà servir la variable global
```

```
    x = 5   # Increment de la variable global en 5
    # Escriure el valor de la variable global
    print("Valor de x des de la funció", x)


x = 20
sa_meva_funcio()   # Escriure el valor de la variable global
print("Variable global x", x)
```

L'ús de variables globals des de les funcions NO se recomana. Alternativament es poden emprar paràmetres i passar els arguments necessaris a les funcions.

14. Gestió d'excepcions

La gestió d'excepcions i errors és un concepte important en la programació. És important perquè, si la nostra aplicació genera un error inesperat, volem tenir la capacitat de controlar-lo i gestionar-lo en lloc de permetre que l'aplicació es bloquegi.

14.1. Ús de ***try-except***

Podem gestionar les excepcions i errors mitjançant les clàusules ***try-except***. La sintaxi bàsica és la següent.

```
try:
    # Codi que pot generar un error o excepció
except ExceptionType:
    # Codi que es crida si es genera una excepció
    # de tipus ExceptionType
```

Aquí tenim un codi que intenta obrir un fitxer i llegir-ne el contingut. Si el fitxer anomenat "*fitxer.txt*" no existeix, l'intèrpret generarà una excepció ***FileNotFoundError***. La clàusula ***except*** capturarà aquesta excepció i imprimirà un missatge que diu "*No s'ha trobat el fitxer*".

```
try:
    with open("fitxer.txt", "r") as f:
        contingut = f.read()
        print(contingut)
except FileNotFoundError:
    print("No s'ha trobat el fitxer.")
```

Podem tenir diverses clàusules ***except*** per gestionar diferents tipus d'excepcions:

```
try:
    # Codi que pot generar un error o excepció
except ExceptionType1:
    # Es crida si es genera una excepció ExceptionType1
except ExceptionType2:
    # Es crida si es genera una excepció ExceptionType2
```

Podem afegir una clàusula ***else*** que s'executa si no es genera cap excepció.

```
try:
    # Codi que pot generar un error o excepció
except ExceptionType:
    # Es crida si es genera una excepció ExceptionType1
else:
    # Es crida si NO es genera cap excepció
```

Finalment, també podem afegir una clàusula ***finally*** que s'executa sempre, sigui quin sigui el resultat del bloc ***try-except***.

```
try:
    # Codi que pot generar un error o excepció
except ExceptionType:
    # Es crida si es genera una excepció ExceptionType
else:
    # Es crida si NO es genera cap excepció
finally:
    # Es crida sempre
```

14.2. Aixecant excepcions manualment

El ***raise*** és una paraula clau que es fa servir per aixecar manualment una excepció. És a dir, permet que el programador generi una excepció en un moment concret del programa.

La sintaxi bàsica és la següent, on ***ExceptionType*** és el tipus d'excepció que volem aixecar i "Missatge d'error" és un missatge que descriu l'error que ha ocorregut.

```
raise ExceptionType("Missatge d'error")
```

Per exemple, imaginem que estem escrivint una aplicació de gestió d'usuari i volem comprovar que la contrasenya que s'ha introduït compleix els requisits de seguretat. Si la contrasenya no compleix aquests requisits, podem generar una excepció amb el raise. Si la contrasenya no els compleix, es genera una excepció ***ValueError*** amb un missatge que descriu l'error. A continuació, a la clàusula ***try***,

cridem la funció *comprova_contrasenya* amb una contrasenya que no compleix els requisits. Com que es genera una excepció, la clàusula ***except*** captura l'excepció i imprimeix un missatge que descriu l'error.

```
def comprova_contrasenya(contrasenya):
    if len(contrasenya) < 8:
        raise ValueError("Ha de tenir mínim 8 caràcters.")
    if not any(car.isupper() for car in contrasenya):
        raise ValueError("Ha de tenir mínim majúscula.")
    if not any(car.isdigit() for car in contrasenya):
        raise ValueError("Ha de tenir mínim un dígit.")

# Exemple d'ús
contrasenya = "secreta1"
try:
    comprova_contrasenya(contrasenya)
except ValueError as error:
    print(f"Error: {error}")
```

14.3. Tipus d'excepcions integrats

Hi ha molts tipus d'excepcions predefinides que es poden utilitzar per manejar situacions anòmales. Alguns són els següents:

Exception	És la classe base per a totes les excepcions. Si volem definir una nova excepció, és recomanable que hereti d'aquesta classe.
ValueError	Es genera quan el valor d'un argument és incorrecte.
TypeError	Es genera quan s'intenta fer una operació amb un tipus de dada incorrecte.
IndexError	Es genera quan s'intenta accedir a un índex que està fora del rang d'una seqüència.
KeyError	Es genera quan s'intenta accedir a una clau que no existeix en un diccionari.
NameError	Es genera quan s'intenta accedir a una variable que no està definida.
FileNotFoundError	Es genera quan s'intenta obrir un fitxer que no existeix.

14.4. Creació de nous tipus d'excepcions

En determinades circumstàncies pot ser que sorgeixi sa necessitat que aixecar un error que no s'ajusti a cap del tipus d'excepcions integrats. En aquests casos es recomana crear excepcions per manejar errors específics del nostre programari.

Per crear una nova excepció personalitzada, s'ha de definir una nova classe que hereti d'***Exception*** o d'una de les seves subclasses. Tot i que de moment no s'ha explicat la programació orientada a objectes, l'hem de emprar en aquesta ocasió i el codi per fer-ho és el següent.

```
class ExcepcioPropia(Exception):
    pass

raise ExcepcioPropia("Això és un error personalitzat!")
```

15. Operacions bàsiques amb arxius

En programació, hi ha dos tipus d'arxius bàsics: els arxius de text i els arxius binaris. La principal diferència, però no l'única, entre els dos és la forma en què s'emmagatzema la informació dins de l'arxiu.

Un arxiu de text és un arxiu que conté caràcters de text, que són interpretats com a caràcters de codi *ASCII* o *Unicode*. Això vol dir que qualsevol editor de text, com el *Bloc de notes* de *Windows* o el *TextEdit* de *Mac*, pot llegir i escriure aquest tipus d'arxius. Els arxius de text s'utilitzen per emmagatzemar informació que es pot llegir i comprendre fàcilment per als humans, com ara text pla, codi font o dades estructurades en format de fitxer *CSV*.

D'altra banda, un arxiu binari és un arxiu que conté dades codificades en un format binari, que no són llegibles directament per als humans. Aquests arxius són fets servir per emmagatzemar informació més complexa, com ara imatges, àudio, vídeo, bases de dades i executables de programari. Els arxius binaris es poden llegir i escriure amb codi específic per aquest tipus d'arxius, i no són llegibles amb un editor de text com el *Bloc de notes*.

15.1. Lectura i escriptura d'arxius de text

Per llegir arxius de text, pots utilitzar la funció open() que retorna un objecte de tipus arxiu. A continuació, pots llegir les dades de l'arxiu amb el mètode read() o llegir línia per línia amb el mètode readline(). Per escriure a un arxiu de text, pots utilitzar el mètode write() en el mateix objecte d'arxiu.

```
# Llegir un arxiu de text
with open("exemple.txt", "r") as f:
    data = f.read()
    print(data)
```

```
# Escriure a un arxiu de text
with open("exemple.txt", "w") as f:
      f.write("Això és un exemple de text que s'escriu a
l'arxiu.")
```

15.2. Lectura i escriptura d'arxius binaris

Per llegir i escriure arxius binaris, pots fer servir el mètode ***open()*** i especificar el mode correcte. Els arxius binaris es llegeixen i escriuen en forma de bytes, i pots utilitzar el mètode ***read()*** per llegir els bytes de l'arxiu i el mètode ***write()*** per escriure bytes en l'arxiu.

```
# Llegir un arxiu binari
with open("exemple.bin", "rb") as f:
    data = f.read()
    print(data)

# Escriure a un arxiu binari
with open("exemple.bin", "wb") as f:
    f.write(b"Exemple de text que s'escriu a l'arxiu.")
```

15.3. Modes d'apertura d'arxius

Aquí tens una taula amb els modes d'obrir un arxiu juntament amb una breu descripció.

r	Obre l'arxiu per llegir-lo. Si l'arxiu no existeix, llança una excepció. Aquest mode és el valor per defecte.
w	Obre l'arxiu per escriure-hi. Si l'arxiu no existeix, es crea un arxiu nou. Si ja existeix un arxiu amb el mateix nom, el contingut anterior s'elimina.
x	Obre l'arxiu per escriure-hi. Si l'arxiu ja existeix, llança una excepció.
a	Obre l'arxiu per afegir-hi contingut. Si l'arxiu no existeix, es crea un arxiu nou. Si ja existeix un arxiu amb el mateix nom, el contingut nou s'afegeix al final de l'arxiu.
b	Modificador que s'utilitza juntament amb altres modes per indicar que s'ha d'obrir l'arxiu en mode binari en lloc de mode de text.

t	Modificador que s'utilitza juntament amb altres modes per indicar que s'ha d'obrir l'arxiu en mode de text en lloc de mode binari.

15.4. Ús de ***with***

És possible treballar amb arxius en Python sense utilitzar el bloc with, però això requereix recordar de tancar l'arxiu explícitament després de cada operació per assegurar que els canvis s'hagin desat correctament.

```
file = open('arxiu.txt', 'r')
content = file.read()
print(content)
file.close()
```

Cal posar molta cura i tancar l'arxiu explícitament amb el mètode ***close()***, en acabar d'emprar l'arxiu, si és que d'una forma responsable, vols anar alliberant els recursos que ja no necessites. El bloc ***with*** proporciona una manera elegant i eficient de treballar amb recursos que han de ser oberts i tancats, com ara arxius.

Quan utilitzem la sintaxi ***with***:

- Tancar l'arxiu automàticament en sortir del bloc. Això fa que el nostre codi sigui més net i llegible.
- Ajuda a prevenir errors en el codi que podrien sorgir si no es tanca l'arxiu adequadament. Si per exemple, es produís un error en el nostre programa abans de tancar manualment l'arxiu, es podria perdre informació important. Amb ***with***, el tancament de l'arxiu és automàtic, independentment de si s'ha produït o no un error.

15.5. Mètodes per operar amb arxius

Aquests són només alguns dels mètodes més comuns per operar amb. N'hi ha molts altres, però aquests són un bon punt de partida per a la majoria de les tasques bàsiques de lectura i escriptura de fitxers.

open()	Obre un arxiu i retorna un objecte fitxer. Aquest mètode admet diversos arguments, com ara la ruta de l'arxiu i el mode en què s'ha d'obrir.
read()	Llegeix el contingut de l'arxiu i el retorna com a cadena de caràcters. Si no s'especifica cap argument, es llegeix tot el contingut de l'arxiu.
readline()	Llegeix una línia de l'arxiu i la retorna com a cadena de caràcters.

write()	Escriu una cadena de caràcters a l'arxiu.
writelines()	Escriu una llista de cadenes de caràcters a l'arxiu.
close()	Tanca l'arxiu. És important tancar l'arxiu després de finalitzar les operacions per assegurar que els canvis s'han desat correctament.
seek()	Estableix la posició del punter de l'arxiu a una posició específica. Això permet llegir o escriure en una posició determinada de l'arxiu.
tell()	Retorna la posició actual del punter de l'arxiu.
flush()	Escriu el contingut de la memòria cau a l'arxiu. Això és útil quan volem assegurar que les dades s'hagin escrit a l'arxiu, per exemple, abans de tancar-lo.

15.6. Errors comuns operant amb arxius

En treballar amb arxius, hi ha diversos errors que poden sorgir i alguns dels errors més comuns són els següents:

- **Error de fitxer no trobat**: Es produeix quan el programa no pot trobar l'arxiu que s'intenta llegir o escriure. Això pot ser causat per un error en el nom del fitxer, la ruta d'accés, la ubicació del fitxer o perquè el fitxer no existeix.

- **Error d'accés denegat**: Es produeix quan el programa no té permisos per llegir o escriure l'arxiu. Això pot ser causat per problemes de permisos d'usuari o perquè l'arxiu està sent utilitzat per un altre programa.

- **Error d'entrada/sortida**: Es produeix quan hi ha problemes en llegir o escriure l'arxiu, com ara la interrupció inesperada del flux d'entrada o sortida, problemes en el disc dur o altres problemes en el sistema de fitxers.

Per a tractar aquests errors, és important incloure un mecanisme de gestió d'errors en el codi. Això implica incloure codi que detecti els errors que puguin sorgir durant l'accés als arxius i un codi amb una estratègia a aplicar en cas de capturar un error.

Exemples d’estratègies a aplicar en cas d’error són les següents

- **Llançar excepcions**: en cas d'error, el codi pot llançar una excepció que serà capturada i tractada en una secció de codi específica. Això permet als programadors indicar al programa què fer en cas d'error, com ara tornar un missatge d'error o intentar una altra operació.

- **Usar instruccions try/except**: aquestes instruccions permeten als programadors intentar una operació i capturar qualsevol error que pugui sorgir en una secció específica de codi. Això proporciona una forma de tractar els errors i prendre accions adequades en cas que sorgeixi un error.

En cas d'error en realitzar operacions d'entrada/sortida sobre arxius el primer és comprovar si l'arxiu existeix a la ruta indicada i el segon verificar si s'han atorgat el permisos d'accés requerits.

15.7. Els arxius CSV

15.8. Els arxius JSON

16. Més sobre els tipus de dades

16.1. Tipus mutables i tipus immutables

16.2. Anotacions (*hints*) de tipus

16.3. Conversió de tipus de dades

16.4. Validació de tipus de dades

17. Les cadenes en profunditat

En Python, les cadenes, també anomenades *strings*, es poden crear utilitzant cometes simples o dobles. Es poden concatenar amb el operador +, o utilitzant el mètode ***join()*** d'una cadena.

```
a = "Hola"
b = "Món"
c = a + " " + b
print(c) # "Hola Món"
```

Per recórrer una cadena, es pot utilitzar un bucle for directament o amb l'auxili del mètode "enumerate" per obtenir l'índex de cada caràcter.

```
frase = "Hola Món"
for caracter in frase:
    print(caracter)
```

17.1. Mètodes de les cadenes

Les cadenes en Python disposen de molts mètodes útils, com ara ***upper()*** i ***lower()*** per convertir els caràcter a majúscules o minúscules respectivament, ***replace()*** per reemplaçar una subcadena i ***strip()*** per eliminar espais en blanc als extrems, entre altres.

```
cadena = "   Hola Món   "
cadena = cadena.strip() # "Hola Món"
cadena = cadena.upper() # "HOLA MÓN"
cadena = cadena.replace("MÓN", "WORLD") # "HOLA WORLD"
```

17.2. Cadenes especials ***f-string***

Les cadenes *f-string* poden utilitzar com a plantilles que permeten incloure variables dins d'una cadena utilitzant *{}*.

```
nom = "Alice"
edat = 25
frase = f"El nom de la persona és {nom} i té {edat} anys"
print(frase) # "El nom de la persona és Alice i té 25 anys"
```

17.3. Cadenes especials ***raw***

Les cadenes *raw*, que són aquelles que no processen caràcters d'escapada, i es defineixen amb *r* davant de les cometes.

```
cadena_normal = "Una línea\nAltre línea"
cadena_raw = r"Una línea\nAltre línea"
print(cadena_normal) # "Una línea
                     # Altre línea"
print(cadena_raw) # "Una línea\nAltre línea"
```

18. Les col·leccions en profunditat

Python té diversos tipus de dades, que són col·leccions de dades més simples, integrades que permeten emmagatzemar i treballar amb col·leccions de dades

Algunes de les principals diferències entre aquests tipus de dades són les següents:

- **Llistes i Tuples**: Les llistes són mutables, és a dir, es poden modificar una vegada creades. Per exemple, podeu afegir, eliminar o canviar elements d'una llista. Les tuples són immutables, és a dir, no es poden modificar un cop creades.

- **Conjunts i Diccionaris**: Els conjunts són una col·lecció d'elements sense ordre i sense duplicats mentre que els diccionaris són una col·lecció d'elements amb un format clau-valor on cada element té una clau única.

- **Llistes, Tuples i Conjunts**: Aquests tipus de dades són similars en què tots emmagatzemen col·leccions d'elements, però les llistes i les tuples són ordenades i les tuples són immutables. Els conjunts no permeten elements duplicats.

18.1. Tipus de col·lecció adequada segons cas d´ús

Les diferents col·leccions tenen característiques que les fan més adequades per a determinades situacions i casos d'ús. Descripció d'alguns exemples de casos d'ús i justificacions per a cada tipus de col·lecció:

- **Llistes**: Són adequades per a casos en què cal emmagatzemar una col·lecció d'elements en un ordre determinat i es vol poder modificar aquests elements després de la seva creació. Per exemple, si voleu emmagatzemar una llista de noms d'estudiants en una classe i voleu ser capaç d'afegir o eliminar estudiants de la llista en el futur, una llista seria una bona opció.

- **Tuples**: Les tuples són adequades per a casos en què cal emmagatzemar una col·lecció d'elements en un ordre determinat, però no es vol ser capaç de modificar aquests elements després de la seva creació. Per exemple, si es vol emmagatzemar un conjunt de coordenades (x, y) i es vol assegurar que aquestes coordenades no canviïn accidentalment en el futur, una tupla seria una bona opció.

- **Conjunts**: Els conjunts són adequats per a casos en què es vol emmagatzemar elements sense ordre i sense elements duplicats. Per exemple, si voleu emmagatzemar un conjunt de números únics d'una llista i voleu evitar números duplicats, un conjunt seria una bona opció.

- **Diccionaris**: Els diccionaris són adequats per a casos en què es vol emmagatzemar elements en un format clau-valor. Per exemple, si voleu

emmagatzemar informació de contacte d'una persona (nom, telèfon, correu electrònic) i voleu ser capaç d'accedir fàcilment a aquesta informació usant les claus "nom", "telèfon" i "correu electrònic", un diccionari seria una bona opció.

18.2. Rendiment de tuples i llistes

Les tuples i les llistes es fan servir per emmagatzemar col·leccions d'elements. No obstant això, hi ha algunes diferències quant a rendiment entre ambdós tipus de dades.

El fet que les tuples són immutables, mentre que les llistes són mutables implica que certes operacions són molt més eficients amb les tuples. Que les tuples siguin immutables significa que una vegada que es crea una tupla, els seus elements no poden ser modificats, mentre que en una llista si es poden modificar. Això fa que les tuples no requereixen l'ús d'operacions addicionals per modificar els seus elements i estan optimitzades permeten un gasto de memòria inferior. Una altra diferència important és que les tuples són més ràpides de recorre que les llistes. Això significa que, en general, les tuples són més ràpides a la recerca d'elements que les llistes, especialment quan es tracta de grans col·leccions de dades.

18.3. Recorregut de col·leccions

A continuació mostren el codi per recórrer els elements de cadascun d'aquests tipus de dades emprant una iteració ***for***.

```
# Llistes
sa_meva_llista = [1, 2, 3, 4, 5]
for element in sa_meva_llista:
    print(element)

# Tuples
mi_tupla = (1, 2, 3, 4, 5)
for element in mi_tupla:
    print(element)

# Conjunts
el_meu_conjunt = {1, 2, 3, 4, 5}
for element in el_meu_conjunt:
    print(element)

# Diccionaris
el_meu_diccionari = {"nom": "Joan", "edat": 30}
for clau, valor in el_meu_diccionari.items():
    print(clau, valor)
```

18.4. Aplicant ***enumerate()*** en els recorreguts

El mètode ***enumerate()*** a Python és una funció que s'utilitza per a iterar sobre una seqüència d'elements, com ara una llista o una cadena de caràcters, i tenir accés a l'índex i al valor de cada element a cada iteració. És molt utilitzat i eficient.

La sintaxi de la funció ***enumerate()*** és:

```
enumerate(iterable, start=0)
```

On ***iterable*** és la seqüència d'elements sobre els quals es vol iterar, com ara una llista o una cadena de caràcters i ***start*** és el número des del qual començarà l'índex dels elements (per defecte és 0)

```
# Recorrer una llista i obtenir l'índice i el valor
# de cada element
lista = ["manzana", "pera", "banana"]
for i, valor in enumerate(lista):
    print(i, valor)
```

18.5. Cerca d'elements a les col·leccions

A continuació, es descriuen algunes maneres de cercar elements en cadascun dels tipus de col·leccions.

- **Llistes**: Una manera comuna de cercar un element en una llista és utilitzant l'operador *in*.

```
sa_meva_llista = [1, 2, 3, 4, 5]
cercat = 4
if cercat in sa_meva_llista:
    print("L'element és a la llista")
else:
    print("L'element no és a la llista")
```

- **Tuples**: El procés de cercar un element en una tupla és similar al de cercar en una llista, s'utilitza l'operador *in* i el procés és el mateix.

```
sa_meva_tupla = (1, 2, 3, 4, 5)
cercat = 4
if cercat in sa_meva_tupla:
    print("L'element és a la tupla")
else:
    print("L'element no és a la tupla")
```

- **Conjunts**: El procés de cercar un element en un conjunt és similar al de cercar en una llista o una tupla, s'usa l'operador in i el procés és el mateix.

```
el_meu_conjunt = {1, 2, 3, 4, 5}
cercat = 4
if cercat in el_meu_conjunt:
    print("L'element està al conjunt")
else:
    print("L'element no està al conjunt")
```

- **Diccionaris**: per cercar un element en un diccionari es pot fer servir la notació d'accés als elements mitjançant claudàtors *[ ]* utilitzant la clau de l'element com a índex. Si la clau existeix, es torna el valor corresponent, si no existeix es torna una excepció ***KeyError***

```
el_meu_diccionari = {"nom": "Joan", "edat": 30}
cercat = "nom"
if cercat in el_meu_diccionari:
    print("La clau existeix al diccionari i el seu valor
és: ", el_meu_diccionari[cercat])
else:
    print("La clau no està al diccionari")
```

19. Mòduls i llibreries

19.1. *Mòduls* integrats

19.2. *Mòduls* externs

Exemples dels més populars i com instal·lar-los.

19.3. Ús del *mòdul* extern ***Colorama***

El mòdul *Colorama* és una eina útil per imprimir text amb diferents colors i estils de format a la consola. Serveix per a millorar la presentació de les sortides de text i fer-les més fàcils de llegir.

Per utilitzar el mòdul, primer s'ha d'instal·lar, normalment a través de la comanda *pip install colorama*. Després, es pot importar el mòdul a l'script de Python i fer servir les funcions que proporciona per imprimir text amb diferents colors i estils de format.

Per imprimir text en vermell, es pot fer servir la funció ***Fore.RED***. La funció ***Fore.RESET*** és important per restablir el color original del text després de la part on volem que sigui diferent.

```
from colorama import Fore

print(Fore.RED + "Això és text en vermell!" + Fore.RESET)
```

20. Recomanacions d'estil de programació

21. Exercicis breus

21.1. amb expressions aritmètiques

1. Calcular el área de un círculo utilizando la fórmula $A = \pi r^2$.

```
pi = 3.14
r = 5
area = pi * (r ** 2)
print(area)
```

2. Calcular el volumen de un cilindro utilizando la fórmula $V = \pi r^2 h$.

```
pi = 3.14
r = 5
h = 10
volumen = pi * (r ** 2) * h
print(volumen)
```

3. Calcular la raíz cuadrada de un número utilizando la función sqrt().

```
import math
num = 25
raiz = math.sqrt(num)
print(raiz)
```

4. Calcular el factorial de un número utilizando un ciclo ***for***.

```
num = 5
factorial = 1
for i in range(1, num + 1):
    factorial *= i
print(factorial)
```

5. Calcular la potencia de un número utilizando el operador exponenciación.

```
base = 2
exponente = 3
potencia = base ** exponente
print(potencia)
```

6. Calcular el porcentaje de un número utilizando el operador multiplicación y división.

```
num = 50
porcentaje = 25
resultado = (num * porcentaje) / 100
print(resultado)
```

7. Calcular el resultado de una división entera utilizando el operador división entera.

```
dividendo = 10
divisor = 3
resultado = dividendo // divisor
print(resultado)
```

8. Calcular el resultado de una división con decimales utilizando el operador de división

```
dividendo = 10
divisor = 3
resultado = dividendo / divisor
print(resultado)
```

9. Calcular el resto de una división utilizando el operador modulo.

```
dividendo = 10
divisor = 3
resultado = dividendo % divisor
print(resultado)
```

10. Calcular el valor absoluto de un número utilizando la función *abs().*

```
num = -5
resultado = abs(num)
print(resultado)
```

21.2. amb expressions lògiques

1. Determinar si un número es par o impar utilizando el operador módulo (%).

```
num = 5
resultado = (num % 2) == 0
print(resultado)
```

2. Determinar si un número está dentro de un rango utilizando los operadores de comparación.

```
num = 15
resultado = (num > 10) and (num < 20)
print(resultado)
```

3. Determinar si una cadena es igual a otra utilizando el operador de igualdad.

```
cadena1 = "Hola"
cadena2 = "Mundo"
resultado = (cadena1 == cadena2)
print(resultado)
```

4. Determinar si una cadena es distinta de otra utilizando el operador de desigualdad.

```
cadena1 = "Hola"
cadena2 = "Mundo"
resultado = (cadena1 != cadena2)
print(resultado)
```

5. Determinar si una lista está vacía utilizando el operador not y la función len().

```
mi_lista = []
resultado = not len(mi_lista)
print(resultado)
```

6. Determinar si un número es positivo o negativo utilizando el operador menor que y el operador mayor que.

```
num = -5
resultado = (num > 0)
print(resultado)
```

7. Determinar si un número es divisible entre 3 utilizando el operador módulo y el operador igualdad.

```
num = 9
resultado = (num % 3) == 0
print(resultado)
```

8. Determinar si una cadena es un número utilizando la función isdigit().

```
cadena = "1234"
resultado = cadena.isdigit()
print(resultado)
```

9. Determinar si una cadena es alfanumérica utilizando la funciónisalnum()

```
cadena = "abcd1234"
resultado = cadena.isalnum()
print(resultado)
```

10. Determinar si una cadena contiene una determinada subcadena utilizando el operador in.

```
cadena = "Hola mundo"
subcadena = "mundo"
resultado = subcadena in cadena
print(resultado)
```

21.3. amb condicions

1. Crear un programa que imprima "Número par" si un número dado es par y "Número impar" si es impar.

```
num = 7
if num % 2 == 0:
    print("Número par")
else:
    print("Número impar")
```

2. Crear un programa que imprima "Aprobado" si un estudiante obtuvo una calificación mayor o igual a 60 y "Reprobado" si es menor.

```
calificacion = 75
if calificacion >= 60:
    print("Aprobado")
else:
    print("Reprobado")
```

3. Crear un programa que imprima "Mayor de edad" si una persona tiene 18 años o más y "Menor de edad" si es menor.

```
edad = 17
if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

4. Crear un programa que imprima "Día de semana" si el día de la semana es lunes, martes, miércoles, jueves o viernes y "Fin de semana" si es sábado o domingo.

```
dia = "sábado"
if dia in ["lunes", "martes", "miércoles", "jueves",
"viernes"]:
    print("Día de semana")
else:
    print("Fin de semana")
```

5. Crear un programa que imprima "Número positivo" si un número dado es positivo y "Número negativo" si es negativo.

```
num = -8
if num >= 0:
    print("Número positivo")
else:
    print("Número negativo")
```

6. Crear un programa que imprima "Letra vocal" si un caracter dado es una vocal y "Letra consonante" si es una consonante.

```
letra = "s"
if letra in ["a", "e", "i", "o", "u"]:
    print("Letra vocal")
else:
    print("Letra consonante")
```

7. Crear un programa que imprima "Número divisible entre 3" si un número dado es divisible entre 3 y "Número no divisible entre 3" si no lo es.

```
num = 6
if num % 3 == 0:
    print("Número divisible entre 3")
else:
    print("Número no divisible entre 3")
```

8. Crear un programa que imprima "Número primo" si un número dado es primo y "Número no primo" si no lo es.

```
num = 11
if all(num % i != 0 for i in range(2, num)):
print("Número primo")
else:
print("Número no primo")
```

9. Crear un programa que imprima "Año bisiesto" si un año dado es bisiesto y "Año no bisiesto" si no lo es.

```
anio = 2020
if (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0):
print("Año bisiesto")
else:
print("Año no bisiesto")
```

10. Crear un programa que imprima "Mes tiene 30 días" si un mes dado tiene 30 días y "Mes tiene 31 días" si tiene 31 días.

```
mes = "febrero"
if mes in ["abril", "junio", "septiembre", "noviembre"]:
print("Mes tiene 30 días")
else:
print("Mes tiene 31 días")
```

21.4. amb iteracions

1. Crear un programa que imprima los números del 1 al 10.

```
for i in range(1,11):
    print(i)
```

2. Crear un programa que imprima los números pares del 1 al 20.

```
for i in range(2,21,2):
    print(i)
```

3. Crear un programa que imprima los números impares del 1 al 20.

```
for i in range(1,21,2):
    print(i)
```

4. Crear un programa que imprima la suma de los números del 1 al 100.

```
suma = 0
for i in range(1,101):
    suma += i
print(suma)
```

5. Crear un programa que imprima los números del 10 al 1.

```
for i in range(10,0,-1):
    print(i)
```

6. Crear un programa que imprima los números del 1 al 20 y su cuadrado.

```
for i in range(1,21):
    print(i, i*i)
```

7. Crear un programa que imprima el factorial de un número dado.

```
num = 5
resultado = 1
for i in range(1,num+1):
    resultado *= i
print(resultado)
```

8. Crear un programa que imprima los números de una lista en orden inverso.

```
numeros = [1,2,3,4,5]
for i in range(len(numeros)-1,-1,-1):
    print(numeros[i])
```

9. Crear un programa que imprima todos los elementos de un diccionario.

```
diccionario = {1: "uno", 2: "dos", 3: "tres"}
for key in diccionario:
    print(key, diccionario[key])
```

10. Crear un programa que imprima los números primos del 1 al 100.

```
for i in range(2,101):
    es_primo = True
    for j in range(2,i):
        if i % j == 0:
            es_primo = False
            break
    if es_primo:
        print(i)
```

22. Mini-projectes

22.1. Optimització d'ús de col·leccions

22.2. Generador de frases

22.3. Camí de sortida d'un laberint